# Agile Models

# What is Agile Software Development?

- Agile: Easily moved, light, nimble, active software processes

- How agility achieved?
  - Fitting the process to the project
  - Avoidance of things that waste time

# Agile Model

- To overcome the shortcomings of the waterfall model of development.
  - Proposed in  mid-1990s
- The agile model was primarily designed:
  - To help projects to adapt to change requests
- In the agile model:
  - The requirements are decomposed into many small incremental parts that can be developed over one to four weeks each.

# Ideology: Agile Manifesto

- Individuals and interactions *over*
  - process and tools
- Working Software *over*
  - comprehensive documentation
- Customer collaboration *over*
  - contract negotiation
- Responding to change *over*
  - following a plan

***http://www.agilemanifesto.org***

# Agile Methodologies

- XP
- Scrum
- Unified process
- Crystal
- DSDM
- Lean

# Agile Model: Principal Techniques

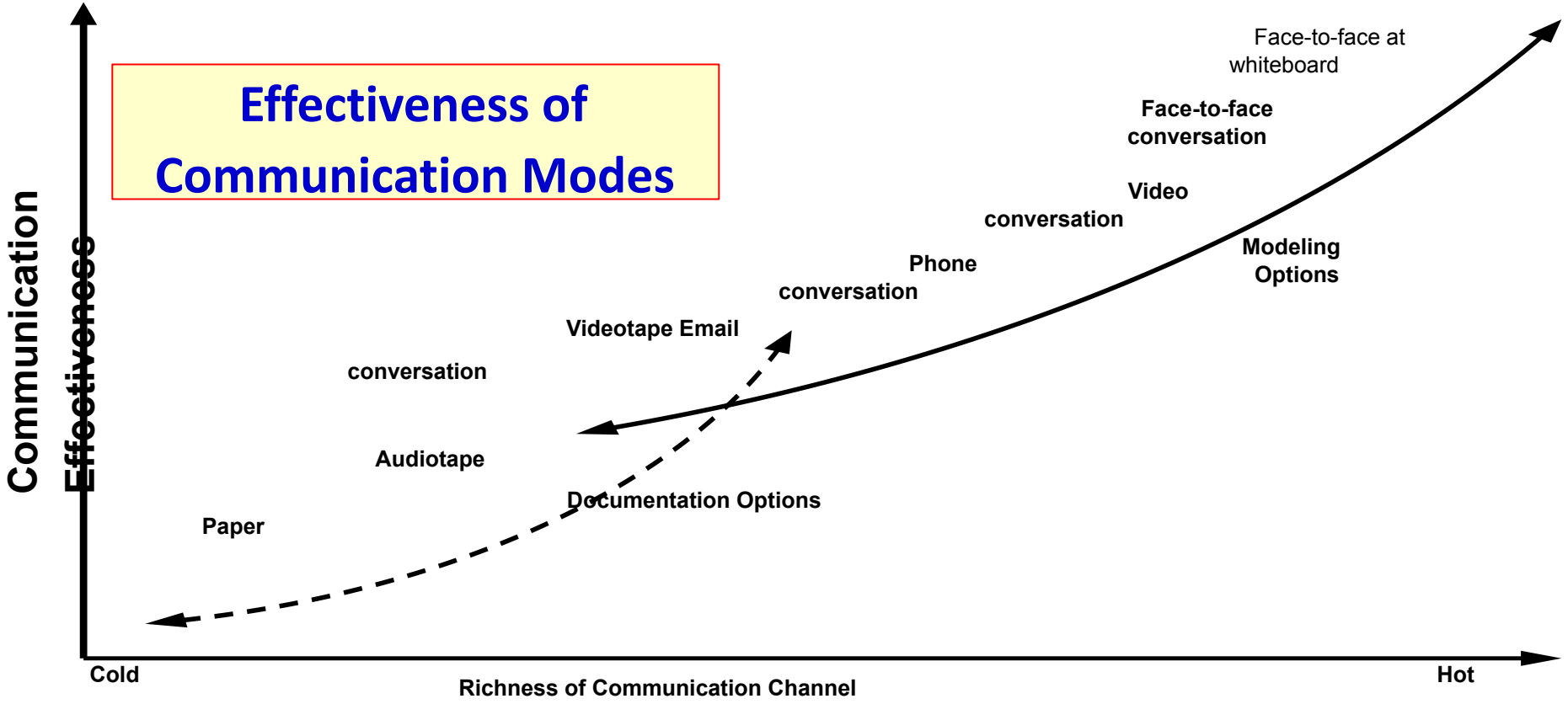
- **User stories:**
  - Simpler than use cases.
- **Metaphors:**
  - Based on user stories, developers propose a common vision of what is required.
- **Spike:**
  - Simple program to explore potential solutions.
- **Refactor:**
  - Restructure code without affecting behavior, improve efficiency, structure, etc.

# Agile Model: Nitty Gritty

- At a time, only one increment is planned, developed, deployed at the customer site.
  - **No long-term plans are made.**
- An iteration may not add significant functionality,
  - But still a new release is invariably made at the end of each iteration
  - Delivered to the customer for regular use.

# Methodology

- **Face-to-face communication favoured over written documents.**
- To facilitate face-to-face communication,
  - Development team to share a single office space.
  - Team size is deliberately kept small (5-9 people)
  - This makes the agile model most suited to the development of small projects.



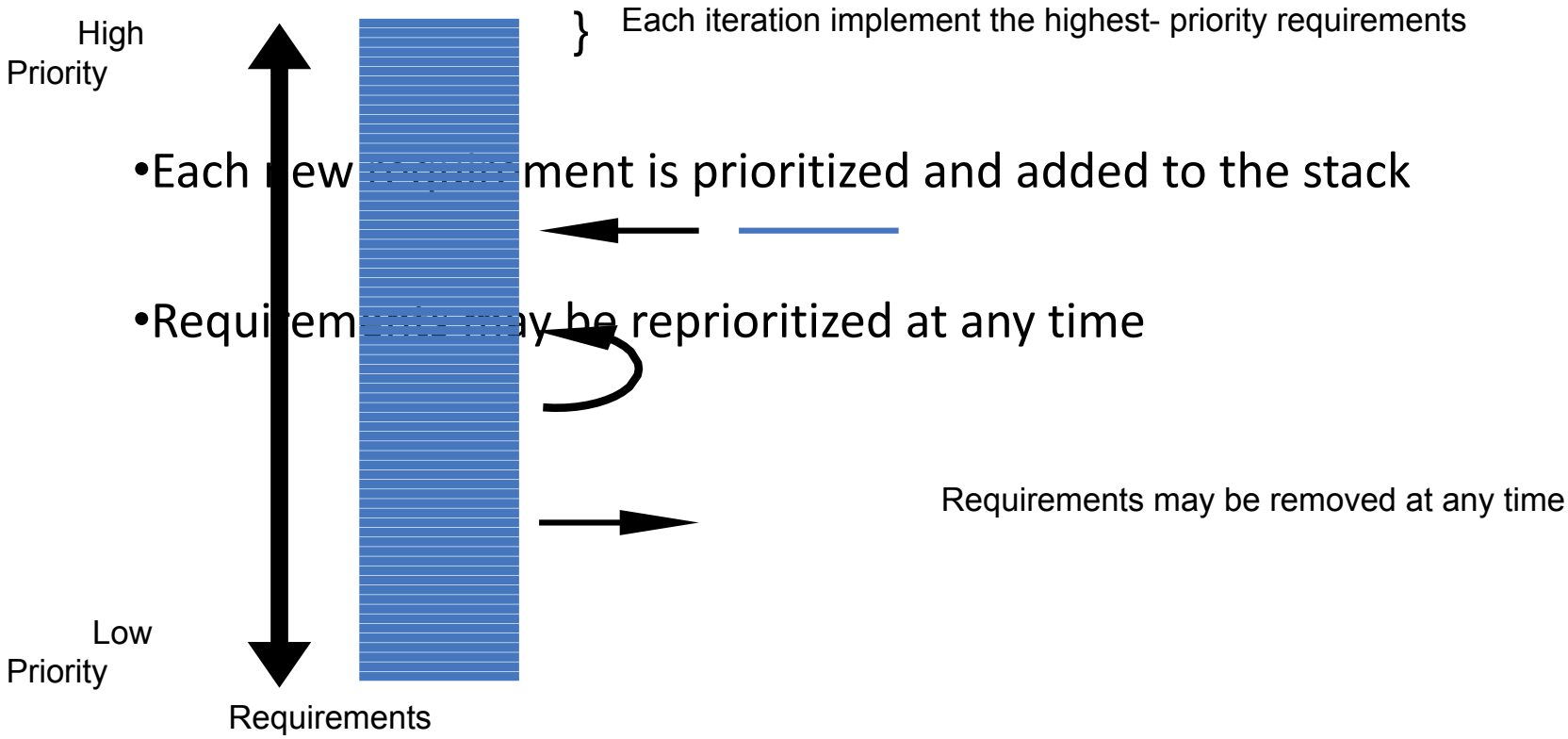
# Agile Model: Principles

- The primary measure of progress:
  - **Incremental release of working software**
- Important principles behind agile model:
  - Frequent delivery of versions --- once every few weeks.
  - Requirements change requests are easily accommodated.
  - Close cooperation between customers and developers.
  - Face-to-face communication among team members.

# Agile Documentation

- Travel light:
  - You need far less documentation than you think.
- Agile documents:
  - Are concise
  - Describe information that is less likely to change
  - Describe “good things to know”
  - Are sufficiently accurate, consistent, and detailed
- Valid reasons to document:
  - Project stakeholders require it
  - To define a contract model
  - To support communication with an external group
  - To think something through

# Agile Software Requirements Management



# Adoption Detractors

- Sketchy definitions, make it possible to have
  - Inconsistent and diverse definitions
- High quality people skills required
- Short iterations inhibit long-term perspective
- Higher risks due to feature creep:
  - Harder to manage feature creep and customer expectations
  - Difficult to quantify cost, time, quality.

# Agile Model Shortcomings

- Derives agility through developing tacit knowledge within the team, rather than any formal document:
  - Can be misinterpreted…
  - External review difficult to get…
  - When project is complete, and team disperses, maintenance becomes difficult…

## Agile Model versus Iterative Waterfall Model

- The waterfall model steps through in a planned sequence:
  - Requirements-capture, analysis, design, coding, and testing .
- Progress is measured in terms of delivered artefacts:
  - Requirement specifications, design documents, test plans, code reviews, etc.
- In contrast agile model sequences:
  - Delivery of working versions of a product in several increments.

## Agile Model versus Iterative Waterfall Model

- As regards to similarity:
  - We can say that Agile teams use the waterfall model on a small scale.

# Agile versus RAD Model

- Agile model does not recommend developing prototypes:
  - Systematic development of each incremental feature is emphasized.
- In contrast:
  - RAD is based on designing quick-and-dirty prototypes, which are then refined into production quality code.

# Agile versus exploratory programming

- Similarity:
  - Frequent re-evaluation of plans,
  - Emphasis on face-to-face communication,
  - Relatively sparse use of documents.
- Agile teams, however, do follow defined and disciplined processes and carry out rigorous designs:
  - This is in contrast to chaotic coding in exploratory programming.

# Extreme Programming (XP)

# Extreme Programming Model

- Extreme programming (XP) was proposed by Kent Beck in 1999.
- The methodology got its name from the fact that:
  - **Recommends taking the best practices to extreme levels.**
  - If something is good, why not do it all the time.

## Taking Good Practices to Extreme

- **If code review is good:**
  - Always review --- **pair programming**
- **If testing is good:**
  - Continually write and execute test cases --- **test-driven development**
- **If incremental development is good:**
  - Come up with new increments every few days
- **If simplicity is good:**
  - Create the simplest design that will support only the currently required functionality.

# Taking to Extreme

- **If design is good,**
  - everybody will design daily (refactoring)
- **If architecture is important,**
  - everybody will work at defining and refining the architecture (metaphor)
- **If integration testing is important,**
  - build and integrate test several times a day (continuous integration)

# 4 Values

- **Communication**:
  - Enhance communication among team members and with the customers.
- **Simplicity**:
  - Build something simple that will work today rather than something that takes time and yet never used
  - May not pay attention for tomorrow
- **Feedback**:
  - System staying out of users is trouble waiting to happen
- **Courage**:
  - Don’t hesitate to discard code

# Best Practices

- **Coding**:
  - without code it is not possible to have a working system.
  - Utmost attention needs to be placed on coding.
- **Testing**:
  - Testing is the primary means for developing a fault-free product.
- **Listening**:
  - Careful listening to the customers is essential to develop a good quality product.

# Best Practices

- **Designing**:
  - Without proper design, a system implementation becomes too complex
  - The dependencies within the system become too numerous to comprehend.
- **Feedback**:
  - Feedback is important in learning customer requirements.

**Extreme Programming Activities**

- **XP Planning**
  - Begins with the creation of “user stories”
  - Agile team assesses each story and assigns a cost
  - Stories are grouped to for a deliverable increment
  - A commitment is made on delivery date
- **XP Design**
  - Follows the KIS principle
  - Encourage the use of CRC cards
  - For difficult design problems, suggests the creation of “spike solutions”—a design prototype
  - Encourages “refactoring”—an iterative refinement of the internal program design

- **XP Coding**
  - Recommends the construction of unit test cases *before* coding commences (test-driven development)
  - Encourages “pair programming”
- **XP Testing**
  - All unit tests are executed daily
  - “Acceptance tests” are defined by the customer and executed to assess customer visible functionalities

**Extreme Programming Activities**

# Full List of XP Practices

1. **Planning** – determine scope of the next release by combining business priorities and technical estimates

2. **Small releases** – put a simple system into production, then release new versions in very short cycles

3. **Metaphor** – all development is guided by a simple shared story of how the whole system works

4. **Simple design** – system is to be designed as simple as possible

5. **Testing** – programmers continuously write and execute unit tests

# Full List of XP Practices

7. **Refactoring** – programmers continuously restructure the system without changing its behavior to remove duplication and simplify

8. **Pair-programming** -- all production code is written with two programmers at one machine

9. **Collective ownership** – anyone can change any code anywhere in the system at any time.

0. **Continuous integration** – integrate and build the system many times a day – every time a task is completed.

# Full List of XP Practices

1. **40-hour week** – work no more than 40 hours a week as a rule
2. **On-site customer** – a user is a part of the team and available full-time to answer questions
3. **Coding standards** – programmers write all code in accordance with rules emphasizing communication through the code

# Emphasizes Test-Driven Development (TDD)

- Based on user story develop test cases
- Implement a quick and dirty feature every couple of days:
  - **Get customer feedback**
  - **Alter if necessary**
  - **Refactor**
- Take up next feature

## Project Characteristics that Suggest Suitability of Extreme Programming

- Projects involving new technology or research projects.
  - In this case, the requirements change rapidly and unforeseen technical problems need to be resolved.
- Small projects:
  - These are easily developed using extreme programming.

# Practice Questions

- What are the stages of iterative waterfall model?
- What are the disadvantages of the iterative waterfall model?
- Why has agile model become so popular?
- What difficulties might be faced if no life cycle model is followed for a certain large project?

# Suggest Suitable Life Cycle Model

- A software for an academic institution to automate its:
  - Course registration and grading
  - Fee collection
  - Staff salary
  - Purchase and store inventory
- The software would be developed by tailoring a similar software that was developed for another educational institution:
  - 70% reuse
  - 10% new code and 20% modification

# Practice Questions

- Which types of risks can be better handled using the spiral model compared to the prototyping model?
- Which type of process model is suitable for the following projects:
  - A customization software
  - A payroll software for contract employees that would be add on to an existing payroll software

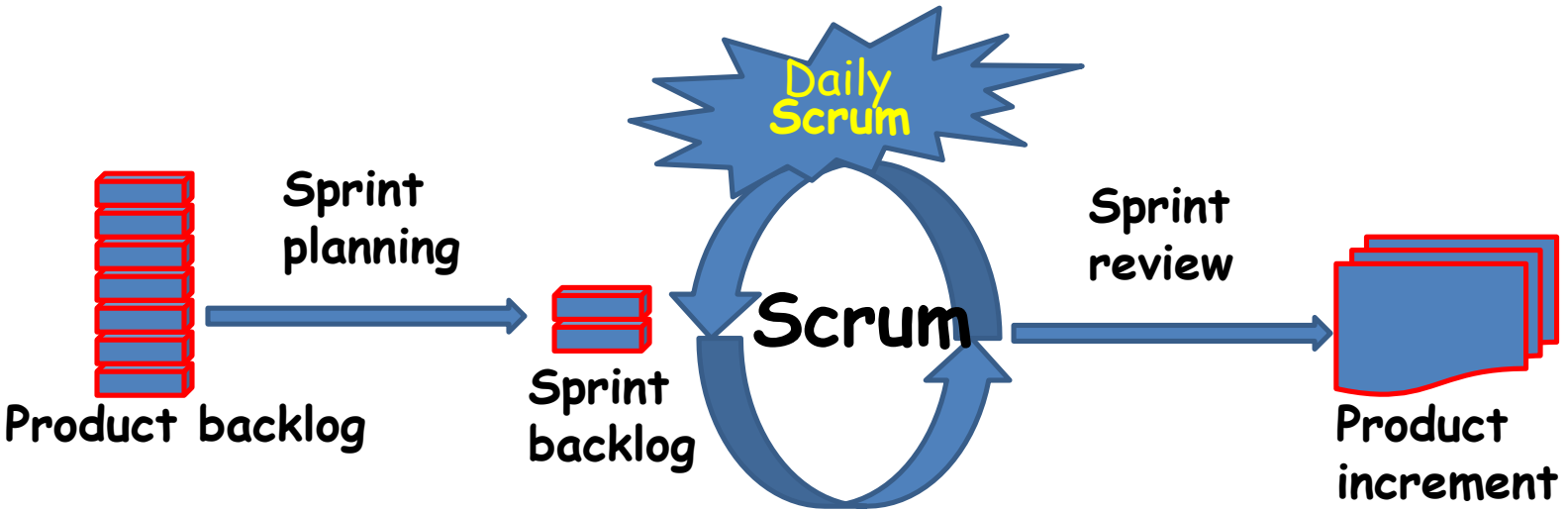
# Practice Questions

- Which lifecycle model would you select for the following project which has been awarded to us by a mobile phone vendor:
  - A new mobile operating system by upgrading the existing operating system
  - Needs to work well efficiently with 4G systems
  - Power usage minimization
  - Directly upload backup data on a cloud infrastructure maintained by the mobile phone vendor

# Scrum

# Scrum: Characteristics

- Self-organizing teams
- Product progresses in a series of month-long **sprints**
- Requirements are captured as items in a list of **product backlog**
- One of the agile processes


Daily
Scrum
Sprint
planning
Sprint
review
Scrum
Product backlog
Sprint
backlog
Product
increment

# Sprint

- Scrum projects progress in a series of “sprints”
  - Analogous to XP iterations or time boxes
  - Target duration is one month
- **Software increment is designed, coded, and tested during the sprint**
- **No changes entertained during a sprint**

# Scrum Framework

- Roles : Product Owner, ScrumMaster, Team
- Ceremonies : Sprint Planning, Sprint Review, Sprint Retrospective, and Daily Scrum Meeting
- Artifacts : Product Backlog, Sprint Backlog, and Burndown Chart

## Key Roles and Responsibilities in Scrum Process

- Product Owner
  - Acts on behalf of customers to represent their interests.
- Development Team
  - Team of five-nine people with cross-functional skill sets.
- Scrum Master (aka Project Manager)
  - Facilitates scrum process and resolves impediments at the team and organization level by acting as a buffer between the team and outside interference.

# Product Owner

- Defines the features of the product
- Decide on release date and content
- Prioritize features according to market value
- Adjust features and priority every iteration, as needed
- Accept or reject work results.

# The Scrum Master

- Represents management to the project
- Removes impediments
- Ensure that the team is fully functional and productive
- Enable close cooperation across all roles and functions
- Shield the team from external interferences

# Scrum Team

- Typically 5-10 people
- Cross-functional
  - QA, Programmers, UI Designers, etc.
- Teams are self-organizing
- Membership can change only between sprints

# Ceremonies

- Sprint Planning Meeting
- Sprint
- Daily Scrum
- Sprint Review Meeting

# Sprint Planning

- Goal is to produce Sprint Backlog
- Product owner works with the Team to negotiate what Backlog Items the Team will work on in order to meet Release Goals
- Scrum Master ensures Team agrees to realistic goals

# Sprint

- Fundamental process flow of mScruf

- A month-long iteration, during which an incremental product functionality completed

- NO outside influence can interfere with the Scrum team during the Sprint

- Each day begins with the Daily Scrum Meeting

- Daily
- 15-minutes
- Stand-up meeting
- Not for problem solving
- Three questions:
  1. What did you do yesterday
  2. What will you do today?
  3. What obstacles are in your way?

# Daily Scrum

## Daily Scrum

- Is NOT a problem solving session
- Is NOT a way to collect information about WHO is behind the schedule
- Is a meeting in which team members make commitments to each other and to the Scrum Master
- Is a good way for a Scrum Master to track the progress of the Team

- Team presents what it accomplished during the sprint
- Typically takes the form of a demo of new features
- Informal
  - 2-hour prep time rule
- Participants
  - Customers
  - Management
  - Product Owner
  - Other engineers

**Sprint Review Meeting**

# Product Backlog

- A list of all desired work on the project
  - Usually a combination of
    - story-based work (“let user search and replace”)
    - task-based work (“improve exception handling”)
- List is prioritized by the Product Owner
  - Typically a Product Manager, Marketing, Internal Customer, etc.

# Product Backlog

- Requirements for a system, expressed as a prioritized list of Backlog Items
  - Managed and owned by Product Owner
  - Spreadsheet (typically)
  - Usually is created during the Sprint Planning Meeting

| | Item # | Description | Est | By |
|---|---|---|---|---|
| **Very High** | | | | |
| | 1 | **Finish database versioning** | 16 | KH |
| | 2 | **Get rid of unneeded shared Java in database** | 8 | KH |
| | - | **Add licensing** | - | - |
| | 3 | Concurrent user licensing | 16 | TG |
| | 4 | Demo / Eval licensing | 16 | TG |
| | | **Analysis Manager** | | |
| | 5 | File formats we support are out of date | 160 | TG |
| | 6 | Round-trip Analyses | 250 | MC |
| **High** | | | | |
| | - | **Enforce unique names** | - | - |
| | 7 | In main application | 24 | KH |
| | 8 | In import | 24 | AM |
| | - | **Admin Program** | - | - |
| | 9 | Delete users | 4 | JM |
| | - | **Analysis Manager** | - | - |
| | 10 | When items are removed from an analysis, they should show up again in the pick list in lower 1/2 of the analysis tab | 8 | TG |
| | - | **Query** | - | - |
| | 11 | Support for wildcards when searching | 16 | T&A |
| | 12 | Sorting of number attributes to handle negative numbers | 16 | T&A |
| | 13 | Horizontal scrolling | 12 | T&A |
| | - | **Population Genetics** | - | - |
| | 14 | Frequency Manager | 400 | T&M |
| | 15 | Query Tool | 400 | T&M |
| | 16 | Additional Editors (which ones) | 240 | T&M |
| | 17 | Study Variable Manager | 240 | T&M |
| | 18 | Haplotypes | 320 | T&M |
| | 19 | **Add icons for v1.1 or 2.0** | - | - |
| | - | **Pedigree Manager** | - | - |
| | 20 | Validate Derived kindred | 4 | KH |
| **Medium** | | | | |
| | - | **Explorer** | - | - |
| | 21 | Launch tab synchronization (only show queries/analyses for logged in users) | 8 | T&A |
| | 22 | Delete settings (?) | 4 | T&A |

**Sample Product Backlog**

# Sprint Backlog

- A subset of Product Backlog Items, which define the work for a Sprint
  - Created by Team members
  - Each Item has it’s own status
  - Updated daily

# Sprint Backlog during the Sprint

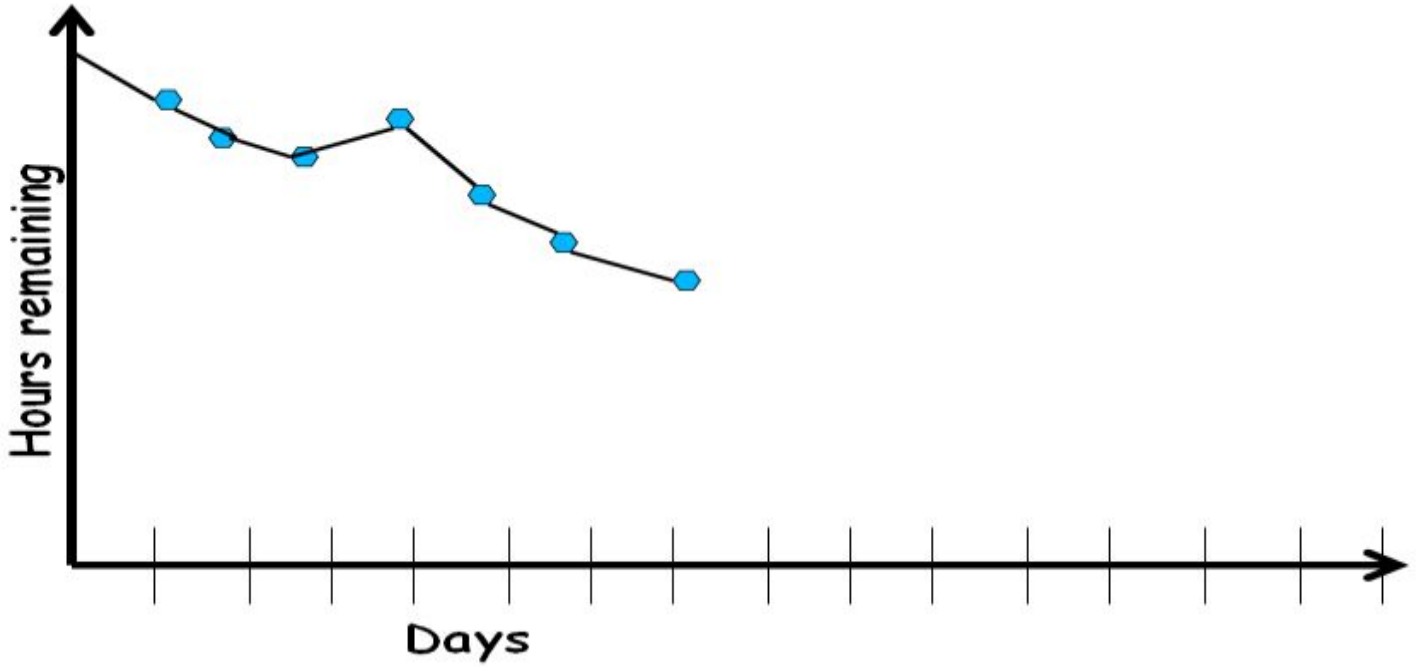
- Changes occur:
  - Team adds new tasks whenever they need to in order to meet the Sprint Goal
  - Team can remove unnecessary tasks
  - But: Sprint Backlog can only be updated by the team
- Estimates are updated whenever there's new information

# Burn down Charts

- Are used to represent “work done”.
- Are remarkably simple but effective Information disseminators
- 3 Types:
  - Sprint Burn down Chart (progress of the Sprint)
  - Release Burn down Chart (progress of release)
  - Product Burn down chart (progress of the Product)

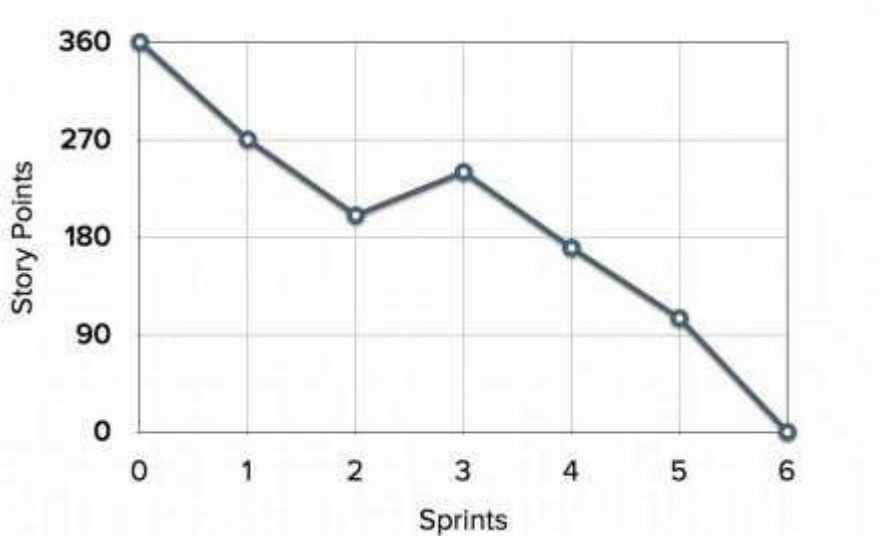
# Sprint Burn down Chart

- Depicts the total Sprint Backlog hours remaining per day
- Shows the estimated amount of time to release
- Ideally should burn down to zero to the end of the Sprint
- Actually is not a straight line


Hours remaining
Days

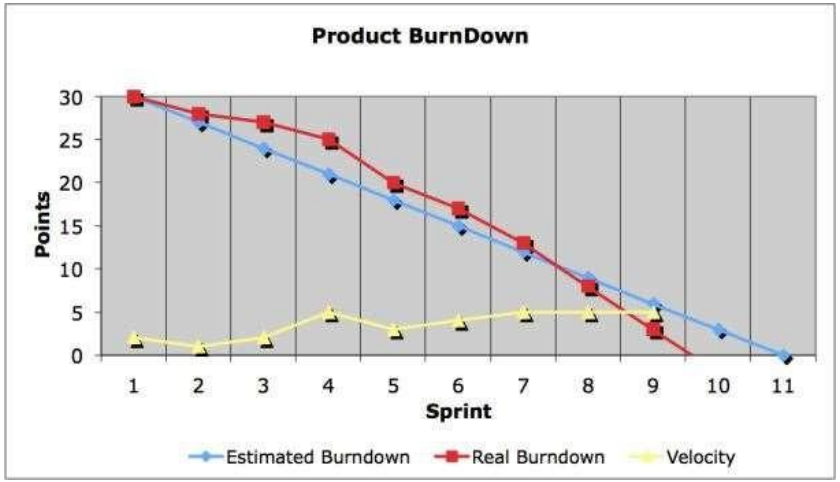
# Release Burndown Chart

- Will the final release be done on right time?
- How many more sprints?
- X-axis: sprints
- Y-axis: amount of story points remaining



# Product Burndown Chart

- Is a “big picture” view of project’s progress (all the releases)



# Scalability of Scrum

- A typical Scrum team is 6-10 people
- Jeff Sutherland - up to over 800 people
- "Scrum of Scrums" or what called "Meta-Scrum“
- Frequency of meetings is based on the degree of coupling between packets

# Writing User story prioritization: Example

As an online user, I can pay the ticket cost online so that I can book my tickets according to my travel plan

Consider stories written for an automated assessment system for evaluation of code(say written in Java) submitted by students in an automated fashion

As a student, I should be able to write classes so that I can solve the problem given during the exam

As a student, I should be able to create .java files so that I can solve the problem given during the exam

As a student, I should be able to compile .java files so that I can solve the problem given during the exam

As a student, I should be able to execute a java program so that I can solve the problem given during the exam

Consider stories written for an automated assessment system for evaluation of code(say written in Java) submitted by students in an automated fashion

As a student, I should be able to write classes so that I can solve the problem given during the exam.

The classes written should follow the coding standards. The access specifiers written against the classes must be checked. The number of instance variables in a class must be checked. The prototype of the methods written need to be verified.

Consider a stories written for an automated assessment system for evaluation of code (say written in Java) submitted by students in an automated fashion.

As a student I should be able to add, update or delete code submitted in the system so that I can debug my code.

| User story # | User story description | INVEST property violated | Revised story |
|---|---|---|---|
| US-01 | As an online user, I can pay the ticket cost online so that I can book my tickets according to my travel plan<br>Solution: A big story with potential of being broken down into several small stories. | Small, Estimable | User story 1: As an online user, I can pay my ticket cost using credit and debit cards of local banks so that I can book my tickets according to my travel plan.<br>User story 2: As an online user, I can pay my ticket cost using credit and debit cards of global banks so that I can book my tickets according to my travel plan. |
| US-02 | Consider stories written for an automated assessment system for evaluation of code(say written in Java) submitted by students in an automated fashion<br>a. As a student, I should be able to write classes so that I can solve the problem given during the exam<br>b. As a student, I should be able to create .java files so that I can solve the problem given during the exam<br>c. As a student, I should be able to compile .java files so that I can solve the problem given during the exam<br>d. As a student, I should be able to execute a java program so that I can solve the problem given during the exam<br>Solution:<br>A bad user story. Stories are too small(resemble subtask) | Small | As a student I should be able to write and execute Java code so that my code can be evaluated |

| User story # | User story description | INVEST property violated | Revised story |
|---|---|---|---|
| US-03 | As a student, I should be able to write classes so that I can solve the problem given during the exam.<br>The classes written should follow the coding standards. The access specifiers written against the classes must be checked. The number of instance variables in a class must be checked. The prototype of the methods written need to be verified.<br>Solution:<br>A bad user story. Too many details are part of the story. Would not encourage conversation with customer. | Negotiable | As a student, I should be able to write classes so that I can solve the problem given during the exam.<br>Constraints for the user story:<br>The classes written should follow the coding standards.<br>The access specifiers written against the classes must be checked.<br>The number of instance variables in a class must be checked.<br>The prototype of the methods written need to be verified. |
| US-04 | As a student I should be able to add, update or delete code submitted in the system so that I can debug my code.<br>Solution: A bad user story. This story is big as the add, update, delete part can be potentially big when implemented. | Estimable, Small | As a student I should be able to add code submitted in the system so that I can debug my code.<br>As a student I should be able to update code submitted in the system so that I can debug my code.<br>As a student I should be able to delete code submitted in the system so that I can debug my code. |

# Writing User story prioritization Exercise

User Story 2

As a researcher, I want to be able to submit research papers online, so that I can track the status of acceptance.

Task

For the above story, arrive at the conditions that can serve as acceptance criteria. On these conditions getting satisfied, the user story can be considered as accomplished.

# Thank You!!

Faculty Name
Department Name

# Agile vs. Plan-Driven Processes

1. Small products and teams:
   - Scalability limited
2. Untested on safety-critical products
3. Good for dynamic, but expensive for stable environments.
4. Require experienced personnel throughout
5. Personnel thrive on freedom and chaos

1. Large products and teams;
   - hard to scale down
2. Proven for highly critical products;
3. Good for stable:
   - But expensive for dynamic environments
4. Require only few experienced personnel
5. Personnel thrive on structure and order